

Object-Oriented Programming

Function & Operator Overloading

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Lecture Outline

01

Function Overloading

Same name, different parameters — compile-time polymorphism

02

Introduction to Operator Overloading

Definition, motivation, and rules

03

Overloading Unary Operators

++, --, unary minus — prefix & postfix

04

Overloading Binary Operators

Arithmetic, comparison, assignment operators

05

Member vs Friend Functions

When to use each approach for operator overloading

06

Overloading << and >> (Stream Operators)

Input/output stream operator overloading

07

Data Type Conversions

Between built-in types and user-defined types

08

Restrictions & Best Practices

What cannot be overloaded and design guidelines

1

Function Overloading

Compile-time polymorphism — same name, different signatures

Feature	Compile-Time Polymorphism	Run-Time Polymorphism
Binding Time	Resolved at compile time (early binding, static binding).	Resolved at runtime (late binding, dynamic binding).
Mechanism	Achieved using method overloading and operator overloading.	Achieved using method overriding (via virtual functions and inheritance).
Flexibility	Less flexible because decisions are fixed at compile time.	More flexible as behavior can adapt to dynamic conditions and user input.
Performance	Generally faster execution since the function to call is known early.	Comparatively slower due to the overhead of dynamic dispatch (determining the correct method to call at runtime).
Inheritance	Does not require inheritance; can occur within a single class.	Involves inheritance, where a child class provides a specific implementation of a parent class method.

What is Function Overloading?

Function Overloading allows multiple functions to have the **same name** but **different parameter lists**. The compiler selects the correct version based on the arguments at the call site.

Different Number of Parameters

```
add(int a, int b)
add(int a, int b, int c)
```

Different Parameter Types

```
area(int side)
area(float radius)
```

Different Order of Parameters

```
show(int x, float y)
show(float y, int x)
```

C++

```
// Three overloaded versions of add()
int add(int a, int b)      { return a+b; }
int add(int a, int b, int c) { return a+b+c; }
double add(double a, double b) { return a+b; }

int main() { cout << add(3,4) << add(1,2,3) << add(1.5,2.5); }
// Output: 7    6    4
```

Function Overloading — Motivation

✗ Without Overloading

```
int  addInt(int a, int b)
{ return a+b; }
float addFloat(float a, float b)
{ return a+b; }
double addDouble(double a, double b)
{ return a+b; }

// Usage — hard to remember names!
addInt(3, 4);
addFloat(1.5f, 2.5f);
addDouble(1.1, 2.2);

// Tedious, error-prone, hard to read
```

✓ With Overloading

```
int  add(int a, int b)
{ return a+b; }
float add(float a, float b)
{ return a+b; }
double add(double a, double b)
{ return a+b; }

// Usage — clean, intuitive!
add(3, 4);
add(1.5f, 2.5f);
add(1.1, 2.2);

// Compiler picks the right one!
```

2

Introduction to Operator Overloading

Giving built-in operators new meaning for user-defined types

What is Operator Overloading?

Operator Overloading is the process of giving the normal C++ operators (+, -, *, ==, <<, etc.) additional meanings when applied to **user-defined data types** (classes).

Without Overloading

```
Date d1(12,3,1989);  
Date d2;  
d2.add_days(d1, 45);  
// complex and obscure!
```



With Overloading

```
Date d1(12,3,1989);  
Date d2;  
d2 = d1 + 45;  
// intuitive and readable!
```

💡 An overloaded operator does NOT lose its original meaning — it gains additional meaning relative to the class it is defined for.

Operators That CAN and CANNOT Be Overloaded

✓ Operators That CAN Be Overloaded

Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Relational	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>
Logical	<code>&&</code> <code> </code> <code>!</code>
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>
Increment/Dec	<code>++</code> <code>--</code>
Stream I/O	<code><<</code> <code>>></code>
Subscript	<code>[]</code>
Call	<code>()</code>
Arrow	<code>-></code> <code>->*</code>
Memory	<code>new</code> <code>delete</code>
Comma	<code>,</code>

✗ CANNOT Be Overloaded

<code>.</code>	Member access
<code>.*</code>	Ptr-to-member
<code>::</code>	Scope resolution
<code>?:</code>	Ternary / conditional
<code>sizeof</code>	Size of operator
<code>typeid</code>	Type ID
<code>static_cast</code>	Type casts
<code>dynamic_cast</code>	

Rules & Restrictions on Operator Overloading

① Precedence CANNOT be changed

Overloaded * still has higher precedence than overloaded +

② Associativity CANNOT be changed

Left-to-right / right-to-left evaluation order is fixed

③ Number of operands CANNOT change

Unary operators stay unary; binary operators stay binary

④ New operators CANNOT be created

You can only overload existing C++ operators

⑤ Overloading is NOT automatic

Overloading + does NOT automatically overload +=

⑥ At least one operand must be a class object

Cannot redefine operators for built-in types only

⑦ = and & work without overloading

Default member-wise copy for = and address-of for &

⑧ () [] -> = must be members

These operators must be implemented as member functions

Operator Overloading — Syntax

Syntax

```
ReturnType operator Symbol (parameters) {  
    // function body  
}
```

// Example: overload + for Distance class

```
Distance operator + (Distance d2) const { // member function — 1 param
```

The operator overloading function can be a friend function or a non-static member function

Member Function

```
Distance operator+(Distance d2);
```

As member function only one parameter for binary and no parameter for unary operator overloading because it uses *this* pointer to get one parameter implicitly

Left operand = calling object (this)

Right operand = parameter

Used for most operator

Friend Function

```
friend Distance operator+(Distance d1,  
Distance d2);
```

As friend function will need both operands parameters

No this pointer

Used when left operand is object of different class or built-in type

Required as Member

```
T& operator=(const T&);  
T& operator[](int);  
T& operator->();
```

() [] -> =
must always be
member functions

3

Overloading Unary Operators

++, --, unary minus — prefix and postfix versions

```
Complex & operator ++()  
{  
    this->real++;  
    this->imaginary++;  
    return *this;  
}
```

Overloading the Increment Operator ++

Code

```
class Counter {  
private:  
    unsigned int count;  
public:  
    Counter() : count(0) {}  
    unsigned int get_count()  
        { return count; }  
    // Prefix: ++c1  
    Counter operator++() {  
        ++count;  
        return Counter(count);  
    }  
};  
  
int main() {  
    Counter c1, c2;  
    ++c1; // c1=1  
    c2 = ++c1; // c1=2, c2=2  
}
```

How It Works

- ① **Keyword operator followed by symbol**
void operator++() or
Counter operator++()
- ② **Return type matters**
If void: ++c1 works but c2=++c1 fails
If Counter: both work
- ③ **Nameless temporary object**
return Counter(count);
Creates unnamed object – efficient
- ④ **Compiler selection**
++intVar → built-in routine
++counterVar → our operator++()

Prefix vs Postfix ++ and --

Prefix: Counter operator++() | **Postfix:** Counter operator++(int) ← *dummy int signals postfix to compiler*

C++

```
class Counter {
    Counter() : count(0) {}
    Counter(int c) : count(c) {} // needed for nameless temp object

    // PREFIX ++c1 - increment THEN return
    Counter operator++ () { return Counter(++count); }

    // POSTFIX c1++ - return THEN increment (dummy int)
    Counter operator++ (int) { return Counter(count++); }
};

int main() {
    Counter c1, c2;           // c1=0, c2=0
    c2 = ++c1;                // PREFIX: c1=1, c2=1 (increments first)
    c2 = c1++;                // POSTFIX: c2=1, c1=2 (returns old value)
    cout << c1.get_count() << c2.get_count(); // Output: 2 1
}
```

4

Overloading Binary Operators

Arithmetic, comparison, assignment and string operators

Overloading + Operator — Distance Class

Code

```
class Distance {
private: int feet; float inches;
public:
    Distance() : feet(0), inches(0.0) {}
    Distance(int ft, float in)
        : feet(ft), inches(in) {}
    void showdist() const {
        cout<<feet<<"'-"<<inches<<'";
    }
    // Overloaded + operator
    Distance operator+(Distance d2) const {
        int f = feet + d2.feet;
        float i = inches + d2.inches;
        if(i >= 12.0){ i-=12.0; f++; }
        return Distance(f, i);
    }
};

int main() {
    Distance d1(3, 6.5), d2(5, 8.0);
    Distance d3 = d1 + d2; // 9'-2.5"
    Distance d4 = d1+d2+d3; // chained!
}
```

Key Points

- Returns Distance object by value — allows chaining
- Left operand = calling object (this)
Right operand = parameter d2
- Nameless temp object Distance(f,i) returned — clean pattern
- const keyword — function does not modify calling object
- d1+d2+d3 works because + returns Distance

Output: d3 = 9'-2.5"
d4 = 18'-5"

Overloading Comparison and Assignment Operators

Comparison Operator < (returns bool)

```
bool operator< (Distance d2) const {  
  
    float bf1 = feet + inches/12;   float bf2 = d2.feet + d2.inches/12;  
  
    return (bf1 < bf2) ? true : false; // usage: if(d1 < d2)
```

Assignment Operator += (void return — modifies self)

```
void operator+= (Distance d2) {  
    feet += d2.feet;   inches += d2.inches;  
    if(inches >= 12.0) { inches -= 12.0; feet++; }  
}  
// usage: d1 += d2; (no return value needed)
```

+ vs +=: operator+ creates a new object and returns it. operator+= modifies the calling object directly (no return). Both must be overloaded separately!

Overloading + for String Concatenation

C++

```
class String {
    enum { SZ=80 };
    char str[SZ];
public:
    String()          { strcpy(str, ""); }
    String(char s[])  { strcpy(str, s); }

    String operator+ (String ss) const {
        String temp;           // make temporary String
        if(strlen(str)+strlen(ss.str) < SZ) {
            strcpy(temp.str, str);    // copy this string
            strcat(temp.str, ss.str); // append argument
        }
        return temp;
    }
};

int main() {
    String s1 = "Hello "; String s2 = "World!";
    String s3 = s1 + s2; // concatenates! Output: Hello World!
}
```



The + operator cannot concatenate C-strings natively. By overloading it in our own String class, we extend C++ to behave like a new, higher-level language.

5

Member vs Friend Functions

Choosing the right implementation for operator overloading

Member Function vs Friend Function — Comparison

Feature	Member Function	Friend Function
Declaration	Inside class body	Inside class with friend keyword
Parameters	Unary: 0 param Binary: 1 param	Unary: 1 param Binary: 2 params
this pointer	✓ Available — left operand	✗ Not available — both explicit
Private access	✓ Direct member access	✓ via friend declaration
Commutativity	✗ $a+b$ may $\neq$ $b+a$	✓ Can implement $a+b = b+a$
Left operand	Must be object of the class	Can be different class or built-in
<< >> operators	✗ Cannot be member (ostream left)	✓ Must be friend function
() [] -> =	✓ MUST be member	✗ Cannot be friend for these

Example — Complex Number + as Member & Friend

Member Function — 1 parameter

```
class Complex {
    double real, imag;
public:
    Complex(double r=0, double i=0)
        : real(r), imag(i) {}
    Complex operator+(Complex c1) {
        return Complex(
            real+c1.real,
            imag+c1.imag);
    }
};
```

Friend Function — 2 parameters

```
class Complex {
    double real, imag;
public:
    Complex(double r=0, double i=0)
        : real(r), imag(i) {}
    friend Complex operator+(Complex a, Complex b){
        return Complex(
            a.real+b.real,
            a.imag+b.imag);
    }
};
```

main.cpp

```
int main() {
    Complex c1(3.0, 4.0), c2(1.5, 2.5);
    Complex c3 = c1 + c2;           // Works the same either way: (4.5 + 6.5i)
    Complex c4 = c1 + c2 + c3;     // Chaining works too!
}
```

6

Overloading Stream & Subscript Operators

<< and >> — *must be non-member (friend) functions*

Overloading [] — *must be non-member (friend) functions*

Overloading << and >> Operators

Why friend? The left operand of << is **ostream** (cout), not our class — so it CANNOT be a member function of our class. It must be a **friend non-member function**.

C++

```
class Distance {
    int feet; float inches;
public:
    Distance(int f=0, float i=0.0) : feet(f), inches(i) {}

    // Overloaded << (output operator)
    friend ostream& operator<< (ostream& os, const Distance& d) {
        os << d.feet << "-" << d.inches << "";
        return os; // return os to allow chaining: cout<<d1<<d2
    }

    // Overloaded >> (input operator)
    friend istream& operator>> (istream& is, Distance& d) {
        cout<<"Enter feet: "; is>>d.feet;
        cout<<"Enter inches: "; is>>d.inches;
        return is;
    }
};

int main() {
    Distance d1, d2(5, 3.5);
    cin >> d1; // uses overloaded >>
    cout << d1 << " " << d2 << endl; // uses overloaded <<
}
```

Overloading Comparison (==) Operator

(1 / 2)

```
// overloaded '==' operator compares strings
#include <iostream>
using namespace std;
#include <string.h>           // for strcmp()
class String{                // user-defined string type
private:
    enum { SZ = 80 };        // size of String objects
    char str[SZ];            // holds a string
public:
    String()                  // constructor, no args
    { strcpy(str, ""); }

    String( char s[] )        // constructor, one arg
    { strcpy(str, s); }

    void display() const      // display a String
    { cout << str; }

    void getstr()              // read a string
    { cin.get(str, SZ); }
```


Overloading Comparison (==) Operator

(2/2)

```
bool operator == (String ss) const{           // check for equality
    return !strcmp(str, ss.str);
}

};

int main()
{
    String s1 = "yes";
    String s2 = "no";
    String s3;
    cout << "\nEnter 'yes' or 'no': ";
    s3.getstr();                               // get String from user
    if(s3==s1)                                 // compare with "yes"
        cout << "You typed yes\n";
    else if(s3==s2)                            // compare with "no"
        cout << "You typed no\n";
    else
        cout << "You didn't follow instructions\n";
    return 0;
}
```

The Subscript Operator ([])

- The subscript operator, [], which is normally used to **access array elements**, can be overloaded.
- This is useful if you want to modify the way arrays work in C++.
 - For example, you might want to make a “**safe**” array: automatically checks the index to ensure that they in bounds.
- Safe array, is implemented by **three** different examples each one using a different approach to **inserting and reading** the array elements:
 - Separate put() and get() functions
 - A single access() function using return by reference
 - The overloaded [] operator using return by reference

Separate getel() and putel() Functions

(1/2)

C++

```
// creates safe array (index values are checked before access)
// uses separate put and get functions
#include <iostream>
#include <process.h>           // for exit()
using namespace std;
const int LIMIT = 100;
class safearray
{
private:
    int arr[LIMIT];
public:
    void putel(int n, int elvalue) // set value of element
    {
        if( n < 0 || n >= LIMIT )
        {
            cout << "\nIndex out of bounds";
            exit(1);
        }
        arr[n] = elvalue;
    }
}
```

Both functions check value of the index number supplied to ensure it's not out of bounds; i.e. less than 0 or larger than the array size (minus 1).

Separate getel() and putel() Functions

(2/2)

C++

```
int getel(int n) const{           // get value of element
    if( n < 0 || n >= LIMIT ){
        cout << "\nIndex out of bounds";
        exit(1);
    }
    return arr[n];
};

int main(){
    safearray sa1;
    for(int j=0; j<LIMIT; j++)    // insert elements
        sa1.putel(j, j*10);
    for(int j=0; j<LIMIT; j++){   // display elements
        int temp = sa1.getel(j);
        cout << "Element " << j << " is " << temp << endl;
    }
    return 0;
}
```

Single access() Function Returning by Reference

(1/2)

C++

```
// uses one access() function for both put and get
#include <iostream>
using namespace std;
#include <process.h>           // for exit()
const int LIMIT = 100;       // array size

class safearray
{
private:
    int arr[LIMIT];

public:
    int& access(int n){
        if( n < 0 || n >= LIMIT ){
            cout << "\n Index out of bounds";
            exit(1);
        }
        return arr[n];
    }
};
```

- We can use the same member function both to insert data into the safe array and to read it out.
- The secret is to return the value from the function by reference.
- This means place the function on the left side of the equal sign, and the value on the right side will be assigned to the variable returned by the function

C++

```
int main()
{
    safearray sa1;

    for(int j=0; j<LIMIT; j++)           // insert elements
        sa1.access(j) = j*10;           // *left* side of equal sign

    for(int j=0; j<LIMIT; j++){           // display elements
        int temp = sa1.access(j);         /*right* side of equal sign
        cout << "Element " << j << " is " << temp << endl;
    }
}
```

Overloading the Subscript Operator []

C++

```
class SafeArray {
    int arr[100];
    int size;
public:
    SafeArray(int s) : size(s) {}

    // Overloaded [] - returns by REFERENCE (can be on left side of =)
    int& operator[] (int index) {
        if(index < 0 || index >= size) {
            cout << "Index out of bounds!"; exit(1);
        }
        return arr[index];    // safe access
    }
};

int main() {
    SafeArray sa(10);
    sa[0] = 42;                // operator[] on LEFT of = (assign)
    int x = sa[0];             // operator[] on RIGHT of = (read)
    sa[99] = 5;                // ⚠ Caught! Index out of bounds
}
```

- To access the safe array using the same subscript ([]) operator that's used for normal C++ arrays, we overload the subscript operator in the safearray class.
- However, since this operator is commonly used on the left side of the equal sign, this overloaded function must return by reference.

Key: The [] operator must return by reference (int&) so it can appear on **both sides** of =. It must be implemented as a **member function** (not a friend).

Complete Example — Distance Class with Multiple Overloaded Operators

interface.h

```
class Distance {
    int feet; float inches;
public:
    Distance(int f=0,float i=0.0):feet(f),inches(i){}
    // Arithmetic: +
    Distance operator+(Distance d2) const {
        int f=feet+d2.feet; float i=inches+d2.inches;
        if(i>=12){i-=12;f++;} return Distance(f,i);
    }
    // Comparison: <
    bool operator<(Distance d2) const {
        return (feet+inches/12) < (d2.feet+d2.inches/12);
    }
    // Assignment: +=
    void operator+=(Distance d2){
        feet+=d2.feet; inches+=d2.inches;
        if(inches>=12){inches-=12;feet++;}
    }
    // Stream: <<
    friend ostream& operator<<(ostream& o, const Distance& d){
        o<<d.feet<<"'-"<<d.inches<<"'"; return o;
    }
};
```

main.cpp

```
int main() {
    Distance d1(3,6.5),d2(5,8.0);
    Distance d3 = d1 + d2;
    d1 += d2;
    if(d1 < d2)
        cout<<"d1 smaller";
    cout<<d1<<" "<<d3;
}
```

9'-2.5" 9'-2.5"

+

Binary add

+=

Assignment

<

Comparison

<<

Stream out

7

Data Type Conversions

Compiler automatically handles conversion between basic types (implicit/explicit)

But between built-in types and user-defined class objects/types???

Type Conversions — Basic ↔ User-Defined Types

Must write routines for such conversions

Basic → User-Defined

Use a one-argument constructor

```
// Constructor converts float to Distance
Distance(float meters) {
    float ff = 3.281 * meters;
    feet    = int(ff);
    inches  = 12*(ff - feet);
}

// Usage:
Distance d = 2.35; // float→Distance
```

User-Defined → Basic

Use a conversion operator function

```
// Conversion operator: Distance→float
operator float() {
    float fracfeet = inches/12;
    fracfeet += float(feet);
    return fracfeet / 3.281;
}

// Usage:
float m = static_cast<float>(d1); or m = d1; // ✓
```

String Class ↔ C-string: Constructor converts **char[] → String**. operator char*() converts **String → C-string**.

```
String(char s[]) { strcpy(str, s); } // char[] → String (constructor)
// String s2 = "Hello!"; OR. s1 = xstr; //where s1 is type String and xstr is a C-string
operator char*() { return str; } // String → char* (conversion op) // xstr != s1
```

Conversion Between C-Strings and String Objects

(1/2)

C++

```
// conversion between ordinary strings and class String
#include <iostream>
using namespace std;
#include <string.h> // for strcpy(), etc.
class String // user-defined string type
{
private:
    enum { SZ = 80 }; // size of all String objects
    char str[SZ]; // holds a C-string
public:
    String() // no-arg constructor
    { str[0] = '\0'; }
    String( char s[] ) // 1-arg constructor
    { strcpy(str, s); } // convert C-string to String
    void display() const // display the String
    { cout << str << endl; }
    operator char*() // conversion operator
    { return str; } // convert String to C-string
};
```

C++

```
{  
  
    String s1;                // use no-arg constructor  
  
    char xstr[] = "Hello World "; // create and initialize C-string  
  
    s1 = xstr;                // use 1-arg constructor to convert C-string to String  
  
    s1.display();             // display String  
  
    String s2 = "Good Morning "; // uses 1-arg constructor to initialize String  
  
    cout << static_cast<char*>(s2); // use conversion operator to convert String to C-string  
                                     //before sending to << op  
  
    cout << endl;  
  
    return 0;  
}
```

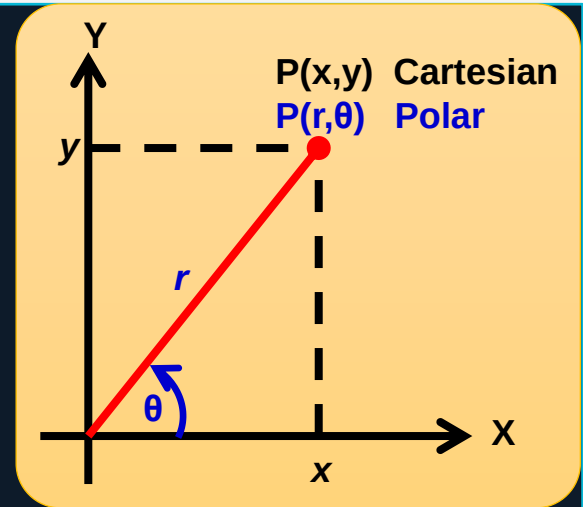
Conversions Between Objects of Different Classes

(1/3)

C++

```
// converts from Polar Coordinates to Cartesian Coordinates
// using 1-Arg constructor in the Cartesian Class
#define M_PI 3.14159265358979323846 //pi
#include <math.h>
#include <iostream>
#include <string>
using namespace std;
class Polar{ // for 2D Polar Coordinates
private:
    double R; // Radius
    double Theta; // Theta in Degrees
public:
    Polar() : R(0), Theta(0){ }

    Polar(double _R, double _Theta) : R(_R), Theta(_Theta) { }
    void Show() const{
        cout << "R = " << R << ", Theta = " << Theta << " Degrees" << endl;
    }
    double Get_R()
    { return R;
    }
}
```



C++

```
double Get_Theta() { return Theta;}
};

class Cartesian{                      // for 2D Cartesian Coordinates
private:
    double X,Y;
public:
    Cartesian() : X(0), Y(0){ }
    Cartesian(double _X, double _Y) : X(_X), Y(_Y) { }

    Cartesian(Polar P){
        double r,t;
        r = P.Get_R();                // Get Radius
        t = P.Get_Theta();            // Get Theta
        t *= M_PI / 180;              // Convert Degrees to Radians
        X = r * cos(t);               // Calculate X
        Y = r * sin(t);               // Calculate Y
    }
}
```

C++

```
void Show() const{
    cout << "X = " << X << ", Y = " << Y << endl;
}
};

int main(){
    Polar Pol(5,53.13);           // R = 5, Theta = 53.13 Degrees
    Cartesian Cat = Pol;         // Convert Polar to Cartesian

    Pol.Show();                  // Show in Polar
    Cat.Show();                  // Show in Cartesian
    return 0;
}
```

Output

```
R = 5, Theta = 53.13 Degrees
X = 3.00001, Y = 3.99999
```

C++

```
// converts from Polar Coordinates to Cartesian Coordinates
// using Operator Overloading in Polar Class
#define M_PI 3.14159265358979323846 //pi
#include <math.h>
#include <iostream>
#include <string>
using namespace std;

class Cartesian{ // for 2D Cartesian Coordinates
private:
    double X,Y;
public:
    Cartesian() : X(0), Y(0){ }
    Cartesian(double _X, double _Y) : X(_X), Y(_Y) { }
    double Get_X(){ return X;}
    double Get_Y(){ return Y;}
    void Show() const{
        cout << "X = " << X << ", Y = " << Y << endl;
    }
};
```


C++

```
class Polar{                                // for 2D Polar Coordinates
private:
    double R;                               // Radius
    double Theta;                           // Theta in Degrees

public:
    Polar() : R(0), Theta(0){ }
    Polar(double _R, double _Theta) : R(_R), Theta(_Theta) { }
    void Show() const{
        cout << "R = " << R << ", Theta = " << Theta << " Degrees" << endl;
    }

    operator Cartesian() const{
        double x, y ,t;
        t = Theta * M_PI / 180 ;           // Converts Degrees to Radians
        x = R * cos(t);
        y = R * sin(t);
        return Cartesian(x,y);
    }
};
```

Conversions Between Objects of Different Classes

(3/3)

C++

```
int main(){
    Polar Pol(5,53.13);           // R = 5, Theta = 53.13 Degrees
    Cartesian Cat = Pol;         // Convert Polar to Cartesian

    Pol.Show();                  // Show in Polar
    Cat.Show();                  // Show in Cartesian

    return 0;
}
```

Output

```
R = 5, Theta = 53.13 Degrees
X = 3.00001, Y = 3.99999
```

Conversion Summary

TABLE 8.2 Type Conversions

	<i>Routine in Destination</i>	<i>Routine in Source</i>
Basic to basic	(Built-In Conversion Operators)	
Basic to class	Constructor	N/A
Class to basic	N/A	Conversion operator
Class to class	Constructor	Conversion operator

When to use what?

- If you have purchased a library of classes, you may not have access to their source code. If you use an object of such a class as the source (polar) in a conversion, you'll have access only to the destination class (cartesian), and you'll need to use a one-argument constructor. (cartesian)
- If the library class object is the destination (cartesian), you must use a conversion operator in the source (polar).

8

Restrictions and Best Practices

*Preventing Conversions with **explicit***

Preventing Conversions with **explicit**

- You have incorporated the appropriate **conversion operator** and **one argument constructor**, but you don't want any other conversion to happen
 - You have to explicitly discourage the conversion that you don't want....
- One solution would be to don't define the operator
 - You may want to construct objects using a single value of another type, but you may not want the implicit conversions a one argument constructor makes possible in other situations. **What to do?**
- The correct solution would be to use **explicit**, It's placed just before the declaration of a one-argument constructor.

C++

```
#include <iostream>
using namespace std;
class Distance{                                // English Distance class
private:
    const float MTF;                          // meters to feet
    int feet;
    float inches;
public:
    // no-args constructor
    Distance() : feet(0), inches(0.0), MTF(3.280833F) { }

    // EXPLICIT one-arg constructor
    explicit Distance(float meters) : MTF(3.280833F)
    {
        float fltfeet = MTF * meters;
        feet = int(fltfeet);
        inches = 12*(fltfeet-feet);
    }
    void showdist()                          // display distance
    { cout << feet << "\'-" << inches << '\\"'; }
};
```

C++

```
int main(){
    void fancyDist(Distance);           // declaration
    Distance dist1(2.35F);              // uses 1-arg constructor to
                                        // convert meters to Distance
    // Distance dist2 = 2.35F;          // ERROR if ctr is explicit

    cout << "\ndist1 = "; dist1.showdist();
    float mtrs = 3.0F;
    cout << "\ndist1 ";

    // fancyDist(mtrs);                  // ERROR if ctor is explicit
    return 0;
}

void fancyDist(Distance d){
    cout << "(in feet and inches) = ";
    d.showdist();
    cout << endl;
}
```

Preventing Conversions with **explicit**

- You can also call fancyDist() with a variable of type float as the argument:

```
fancyDist(mtrs);
```

- The compiler will realize it's the wrong type and look for a conversion operator.
 - Finding a Distance constructor that takes type float as an argument, it will arrange for this constructor to convert float to Distance and pass the Distance value to the function.
- This is an implicit conversion, one which you may not have intended to make possible.
- Making the constructor explicit, we prevent implicit conversions.
- As a side effect of the explicit constructor, you can't use the form of object initialization that uses an equal sign i.e. Distance dist1 = 2.35F;
- whereas the form with parentheses works i.e. Distance dist1(2.35F);

Best Practices

✓ DO — Best Practices

- Use overloading to make code intuitive — only when operator meaning is obvious
- Return `*this` from assignment operators to enable chaining
- Return by const reference from `<<` to allow chaining
- Overload related operators together: if `+` then also `+=`
- Use `const` keyword on operators that don't modify the object
- Prefer member functions; use friend only when needed (e.g. `<<` `>>`)

✗ DON'T — Common Mistakes

- Overload operators with confusing semantics (e.g. `+` to subtract)
- Overload operators just to be clever — readability is key
- Change the fundamental meaning of an operator for a class
- Forget that overloading `+` does NOT automatically overload `+=`
- Use friend functions when a member function works fine
- Forget to handle self-assignment in `operator=` (check this `!= &rhs`)

Lecture Summary — Operator Overloading in C++

Function Overload

Same function name, different parameter lists. Compiler selects correct version at compile time.

Operator Overload

Give built-in operators new meaning for user-defined types. Resembles function overloading.

Syntax

`ReturnType operator Symbol (params) { }` — keyword operator followed by the operator symbol.

Unary

`++ --`

Prefix: `operator++()` | Postfix: `operator++(int)` — dummy int distinguishes them.

Binary

`+ < +=`

Left operand = object (member func). Returns new object for `+`, `bool` for `<`, `void` for `+=`.

Member vs Friend

Member: 1 less param, uses `this`. Friend: all params explicit, needed for `<<` and `>>`.

Thank You

Function & Operator Overloading · Object-Oriented Programming in C++